

5.8.27

Jnanesh Sathisha karmar - EE25BTECH11029

October 12,2025

Question:

$\mathbf{A} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & -2 & 4 \end{pmatrix}$ and $\mathbf{I} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$ and $\mathbf{A}^{-1} = \frac{1}{6}(\mathbf{A}^2 + c\mathbf{A} + d\mathbf{I})$, then value of c and d are

① $(-6, -11)$

② $(6, 11)$

③ $(-6, 11)$

④ $(6, -11)$

Theoretical Solution

The characteristic equation for **A** is given by $\det(\mathbf{A} - \lambda \mathbf{I}) = 0$.

First, we construct the matrix $(\mathbf{A} - \lambda \mathbf{I})$:

$$\mathbf{A} - \lambda \mathbf{I} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & -2 & 4 \end{pmatrix} - \lambda \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 - \lambda & 0 & 0 \\ 0 & 1 - \lambda & 1 \\ 0 & -2 & 4 - \lambda \end{pmatrix} \quad (1)$$

Next, we calculate its determinant:

$$\begin{aligned} \det(\mathbf{A} - \lambda \mathbf{I}) &= (1 - \lambda) \begin{vmatrix} 1 - \lambda & 1 \\ -2 & 4 - \lambda \end{vmatrix} \\ &= (1 - \lambda) [(1 - \lambda)(4 - \lambda) - (1)(-2)] \\ &= (1 - \lambda) [4 - 4\lambda - \lambda + \lambda^2 + 2] \\ &= (1 - \lambda) (\lambda^2 - 5\lambda + 6) \\ &= \lambda^2 - 5\lambda + 6 - \lambda^3 + 5\lambda^2 - 6\lambda \\ &= -\lambda^3 + 6\lambda^2 - 11\lambda + 6 \end{aligned}$$

Theoretical Solution

Setting the determinant to zero gives the characteristic equation.

Multiplying by -1 for convention, we get:

$$\lambda^3 - 6\lambda^2 + 11\lambda - 6 = 0 \quad (2)$$

The theorem allows us to substitute the matrix A for λ in the characteristic equation. The constant term is multiplied by the identity matrix I .

$$\mathbf{A}^3 - 6\mathbf{A}^2 + 11\mathbf{A} - 6\mathbf{I} = \mathbf{O} \quad (3)$$

where $\mathbf{O}$ is the zero matrix.

We can rearrange Equation (4) to solve for the inverse. First, move the identity term to the right side:

$$\mathbf{A}^3 - 6\mathbf{A}^2 + 11\mathbf{A} = 6\mathbf{I} \quad (4)$$

Now, multiply the entire equation by $\mathbf{A}^{-1}$ from the left:

$$\mathbf{A}^{-1} (\mathbf{A}^3 - 6\mathbf{A}^2 + 11\mathbf{A}) = \mathbf{A}^{-1} (6\mathbf{I})$$

$$\mathbf{A}^2 - 6\mathbf{A} + 11\mathbf{I} = 6\mathbf{A}^{-1}$$

Theoretical Solution

Finally, isolate $\mathbf{A}^{-1}$ by dividing by 6:

$$\mathbf{A}^{-1} = \frac{1}{6} (\mathbf{A}^2 - 6\mathbf{A} + 11\mathbf{I}) \quad (5)$$

We now compare the derived expression (5) with the expression given in the question.

Derived Expression: $\mathbf{A}^{-1} = \frac{1}{6} (\mathbf{A}^2 - 6\mathbf{A} + 11\mathbf{I})$

Given Expression: $\mathbf{A}^{-1} = \frac{1}{6} (\mathbf{A}^2 + c\mathbf{A} + d\mathbf{I})$

By directly comparing the coefficients of the matrices $\mathbf{A}$ and $\mathbf{I}$ inside the parentheses, we find:

$$c = -6 \quad \text{and} \quad d = 11 \quad (6)$$

The solution is therefore third option, $(c, d) = (-6, 11)$.

C Code

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <math.h>

#define N 3

void printMatrix(double mat[N][N], const char* name);
void multiplyMatrices(double mat1[N][N], double mat2[N][N],
    double result[N][N]);
bool invertMatrix(double mat[N][N], double inverse[N][N]);
void scalarMultiply(double scalar, double mat[N][N], double
    result[N][N]);
void subtractMatrices(double mat1[N][N], double mat2[N][N],
    double result[N][N]);

int main() {
    double A[N][N] = {
        {1.0, 0.0, 0.0},
```

```
printf("--- Initial Matrix A ---\n");
printMatrix(A, "A");
multiplyMatrices(A, A, A_squared);
printf("\n--- Step 1: Calculated A^2 ---\n");
printMatrix(A_squared, "A^2");
printf("\n--- Step 2: Calculating A^-1 ---\n");
if (!invertMatrix(A, A_inverse)) {
    printf("Matrix A is not invertible. Cannot solve.\n");
    return 1;}
printMatrix(A_inverse, "A^-1");
printf("\n--- Step 3: Calculating RHS = 6*A^-1 - A^2 ---\n");
scalarMultiply(6.0, A_inverse, temp_matrix);
printMatrix(temp_matrix, "6 * A^-1");
subtractMatrices(temp_matrix, A_squared, RHS_matrix);
printMatrix(RHS_matrix, "RHS Matrix");
double c = RHS_matrix[1][2];
double d = RHS_matrix[1][1] - c;
```

```

    printf("\n--- Step 4: Solved for c and d ---\n");
    printf("From element (2, 3) of the matrix equation, we get c
        = %.2f\n", c);
    printf("From element (2, 2), we get c + d = %.2f, so d = %.2f
        - %.2f = %.2f\n", RHS_matrix[1][1], RHS_matrix[1][1], c,
        d);
    printf("\n===== \n");
    printf(" Final Solution \n");
    printf("===== \n");
    printf("c = %.2f\n", c);
    printf("d = %.2f\n", d);
    printf("===== \n");
    return 0;}

void printMatrix(double mat[N][N], const char* name) {
    printf("Matrix %s:\n", name);
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {printf("%8.3f ", mat[i][j])
            ;}
        printf("\n");
    }
}

```

C Code

```
void multiplyMatrices(double mat1[N][N], double mat2[N][N],
    double result[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            result[i][j] = 0;
            for (int k = 0; k < N; k++) {
                result[i][j] += mat1[i][k] * mat2[k][j];
            }
        }
    }
}
```

```
bool invertMatrix(double mat[N][N], double inverse[N][N]) {
    double augvec[N][N * 2];

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            augvec[i][j] = mat[i][j];
        }
    }
}
```

```
for (int i = 0; i < N; i++) {  
    double pivot = augvec[i][i];  
  
    if (pivot == 0) {  
        int max_row = i;  
        for (int k = i + 1; k < N; k++) {  
            if (fabs(augvec[k][i]) > fabs(augvec[max_row][i]))  
            {  
                max_row = k;  
            }  
        }  
  
        if (max_row != i) {  
            for (int j = 0; j < 2 * N; j++) {  
                double temp = augvec[i][j];  
                augvec[i][j] = augvec[max_row][j];  
                augvec[max_row][j] = temp;  
            }  
        }  
    }  
}
```

C Code

```
    pivot = augvec[i][i];
    if (pivot == 0) return false;
    for (int j = 0; j < 2 * N; j++) {augvec[i][j] /= pivot;}
    for (int k = 0; k < N; k++) {
        if (k != i) {
            double factor = augvec[k][i];
            for (int j = 0; j < 2 * N; j++) {
                augvec[k][j] -= factor * augvec[i][j];}
        }
    }
}
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        inverse[i][j] = augvec[i][j + N];
    }
}
return true;
}
```

```
void scalarMultiply(double scalar, double mat[N][N], double
    result[N][N]) {
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        result[i][j] = mat[i][j] * scalar;
    }
}

}

void subtractMatrices(double mat1[N][N], double mat2[N][N],
    double result[N][N]) {
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        result[i][j] = mat1[i][j] - mat2[i][j];
    }
}

}
```